



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación general de pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad curricular: Lenguajes y compiladores

Semestre 7 – Sección: 1

LENGUAJES Y GRAMATICAS FORMALES

Profesor:
Félix, Marquez.

Alumnos:
Daniela, Longart. C.I: 31.445.710.
Fabiana, Martínez. C.I: 30.498.516.
Sebastián, Ortiz. C.I: 30.576.350.
Haddan, Valencia. C.I: 31.181.222.

Ciudad Guayana 17 de junio del 2026.

RESUMEN ACADEMICO

El trabajo presenta una investigación teórico-práctica sobre los fundamentos de los Lenguajes Formales y las Gramáticas en el contexto de la ingeniería de compiladores.

El objetivo central es doble: fundamentar rigurosamente los mecanismos de generación sintáctica y aplicar dichos modelos abstractos a la resolución de problemas específicos y optimización de código.

La investigación se estructura en cuatro componentes esenciales:

- **Fundamentos y Clasificación:** Se deconstruye la relación matemática entre gramáticas y lenguajes mediante el proceso de derivación formal. Se examinan los cuatro niveles de la Jerarquía de Chomsky (Tipo 0 a Tipo 3), definiendo sus restricciones de producción y formalizando sus estructuras mediante la notación BNF (Backus-Naur Form).
- **Modelado y Derivación Práctica:** Se diseña una Gramática Libre de Contexto (GLC) restrictiva sobre el alfabeto $\Sigma = \{a, c, g, t\}$ orientada al modelado geométrico y de herramientas de dibujo. Se desarrollan cinco casos de estudio de derivaciones sucesivas para validar la generación de estructuras complejas (como cuadrados o árboles).
- **Higiene y Optimización Gramatical:** Se abordan las patologías estructurales que degradan el rendimiento de un analizador sintáctico. Se ejemplifica y resuelve de forma algorítmica la ambigüedad (mediante árboles de derivación múltiples), la recursividad por la izquierda y la necesidad de factorización por la izquierda, presentando las gramáticas optimizadas resultantes.
- **Reconocimiento de Patrones y Autómatas:** Se traslada la teoría de lenguajes regulares al ámbito del análisis léxico mediante la simplificación de la notación PGN (Portable Game Notation) para ajedrez. Se formulan las expresiones regulares (Regex) y se construyen los Autómatas Finitos Determinísticos (AFD) equivalentes para el reconocimiento exacto de movimientos válidos.

INTRODUCCIÓN

El estudio contemporáneo de los lenguajes de programación y sus procesadores requiere trascender la mera observación empírica de sus estructuras superficiales. Si bien la aproximación práctica permite comprender los paradigmas operativos y las morfologías léxicas o sintácticas básicas, el diseño y la construcción robusta de compiladores e intérpretes exigen un dominio matemático profundo de los mecanismos de generación y reconocimiento de cadenas. En este sentido, la Teoría de Lenguajes y Gramáticas Formales provee el marco conceptual abstracto indispensable para modelar, validar y traducir cualquier lenguaje estructurado L.

A nivel atómico, la construcción de un lenguaje formal se fundamenta en estructuras matemáticas discretas: un alfabeto finito de símbolos cuyas combinaciones lógicas dan lugar a palabras o cadenas válidas bajo reglas sintácticas estrictas. El motor operativo de este sistema son las gramáticas formales, inspiradas en la jerarquía clasificatoria propuesta por Noam Chomsky en la década de 1950.

Esta jerarquía organiza los lenguajes en cuatro niveles según la complejidad de sus reglas de producción y la capacidad computacional de los autómatas requeridos para su reconocimiento, trazando una línea clara entre la eficiencia del análisis léxico mediante expresiones regulares y la sofisticación del análisis sintáctico mediante gramáticas libres de contexto.

El presente informe de investigación aborda de manera sistemática los fundamentos teóricos y las aplicaciones prácticas de esta jerarquía. Se analizan el mecanismo de derivación gramatical, el modelado formal aplicado a la generación de gráficos mediante restricciones alfabéticas, y los procedimientos de optimización e higiene gramatical esenciales para prevenir patologías como la ambigüedad o la recursividad infinita en los parsers.

Finalmente, se extiende el estudio hacia los autómatas finitos mediante el diseño de motores de reconocimiento basados en la notación PGN de ajedrez, demostrando que el pensamiento computacional riguroso es la piedra angular para transformar al ingeniero en un arquitecto de la informática moderna.

DESARROLLO

1. Relación Gramática-Lenguaje

La gramática formal es un conjunto finito de reglas que permite generar cadenas (secuencias de símbolos) pertenecientes a un lenguaje formal. El lenguaje generado por una gramática es el conjunto de todas las cadenas compuestas exclusivamente por símbolos terminales (el alfabeto del lenguaje) que pueden obtenerse a partir del símbolo inicial aplicando repetidamente las reglas de producción.

El mecanismo de generación es la derivación: se parte del símbolo inicial y, en cada paso, se reemplaza un símbolo no terminal (variable) por el lado derecho de una regla que lo tenga como lado izquierdo. El proceso termina cuando ya no quedan símbolos no terminales; la cadena resultante es una palabra del lenguaje.

En resumen:

- Gramática: especificación finita y generativa.
- Lenguaje: conjunto (posiblemente infinito) de todas las cadenas derivables.

Ejemplo práctico paso a paso

Consideremos la gramática (G) definida por:

- Terminales ($T = \{a, b\}$)
- No terminales ($V = \{S\}$)
- Símbolo inicial (S)
- Reglas de producción:
 - ($S \Rightarrow aSb$)
 - ($S \Rightarrow ab$)

Esta gramática genera el lenguaje ($L(G) = \{ a^n b^n \mid n \geq 1 \}$), es decir, cadenas con igual número de (a)'s seguidos de igual número de (b)'s.

Derivación paso a paso para la cadena aaabbb ($n = 3$)

Paso	Cadena actual	Regla aplicada	Explicación
1	(S)	(inicio)	Partimos del símbolo inicial.
2	(aSb)	($S \Rightarrow aSb$)	Reemplazamos (S) por (aSb).
3	($aaSbb$)	($S \Rightarrow aSb$)	Reemplazamos el (S) interior por (aSb) otra vez.

4	(aaabbb)	(S => ab)	Reemplazamos el último (S) por (ab). Ya no hay no terminales.
---	------------	-------------	---

La derivación se escribe formalmente como: [S => aSb => aaSbb => aaabbb]

La cadena aaabbb pertenece al lenguaje porque consta solo de terminales y se obtuvo siguiendo las reglas.

Árbol de derivación

```

      S
     /\
    a S b
     /\
    a S b
     /\
    a b

```

Cada rama respeta una regla de producción.

¿Qué lenguaje genera exactamente?

- Aplicando la regla 1 (k) veces y luego la regla 2 una vez, se obtienen (k+1) letras (a) seguidas de (k+1) letras (b).
- Por lo tanto: $L(G) = \{ a^n b^n \mid n \geq 1 \}$.

Tipo 0 – Gramáticas sin restricciones (lenguajes recursivamente enumerables)

Definición: No hay restricciones sobre las producciones. Toda regla es de la forma $\alpha \rightarrow \beta$, donde α, β son cadenas de terminales y no terminales, con α (expto posiblemente $S \rightarrow \epsilon$ si S no aparece en el lado derecho de ninguna regla).

Lenguaje generado: Lenguajes recursivamente enumerables (reconocibles por una máquina de Turing).

Ejemplo práctico en notación BNF extendida (permitiendo múltiples símbolos en el lado izquierdo):

Gramática que genera $L = \{ a^n b^n \mid n \geq 1 \}$

```

<S> ::= <A><B><C>
<A> ::= a<A> | a
<B> ::= b<B> | b

```

$$\begin{aligned}
\langle C \rangle &::= c \langle C \rangle \mid c \\
\langle A \rangle \langle B \rangle &::= \langle A \rangle \langle X \rangle \langle B \rangle \\
\langle X \rangle \langle B \rangle &::= \langle B \rangle \langle X \rangle \\
\langle X \rangle \langle C \rangle &::= \langle Y \rangle \langle C \rangle \\
\langle Y \rangle \langle C \rangle &::= \langle C \rangle \langle Y \rangle \\
\langle A \rangle \langle Y \rangle &::= \langle A \rangle \mid a \\
\langle B \rangle \langle Y \rangle &::= \langle B \rangle \mid b \\
\langle C \rangle \langle Y \rangle &::= \langle C \rangle \mid c
\end{aligned}$$

Tipo 1 – Gramáticas sensibles al contexto (lenguajes sensibles al contexto)

Definición: Toda producción es de la forma $\alpha A \beta \rightarrow \alpha \gamma \beta$ $\alpha A \beta \rightarrow \alpha \gamma \beta$, donde A es un no terminal, α, β, γ son cadenas ($\gamma \neq \varepsilon$), y además se permite $S \rightarrow \varepsilon$ si SS no aparece en la derecha de ninguna regla. Equivalente a gramáticas no contractivas ($|\alpha| \leq |\beta|$).

Lenguaje generado: Lenguajes sensibles al contexto (reconocibles por autómatas linealmente acotados).

Ejemplo práctico en notación BNF extendida (con contexto):

Gramática para el mismo lenguaje $L = \{a^n b^n c^n \mid n \geq 1\}$ usando reglas no contractivas.

$$\begin{aligned}
\langle S \rangle &::= a \langle S \rangle \langle C \rangle \mid a \langle B \rangle \langle C \rangle \\
\langle B \rangle \langle C \rangle &::= \langle B \rangle \langle D \rangle \\
\langle D \rangle \langle C \rangle &::= \langle C \rangle \langle D \rangle \\
\langle C \rangle \langle D \rangle &::= \langle D \rangle \langle C \rangle \quad (* \text{ para ordenar } *) \\
\langle C \rangle \langle B \rangle &::= \langle B \rangle \langle C \rangle \\
a \langle B \rangle &::= a b \\
b \langle B \rangle &::= b b \\
b \langle C \rangle &::= b c \\
c \langle C \rangle &::= c c
\end{aligned}$$

Las reglas con dos símbolos en el lado izquierdo muestran el contexto necesario para respetar el orden. Esta gramática genera exactamente $a^n b^n c^n$.

Tipo 2 – Gramáticas libres de contexto (lenguajes libres de contexto)

Definición: Toda producción es de la forma $A \rightarrow \gamma A \rightarrow \gamma$, donde AA es un no terminal y $\gamma\gamma$ es una cadena de terminales y no terminales (puede ser $\epsilon\epsilon$). No hay contexto alrededor de AA .

Lenguaje generado: Lenguajes libres de contexto (reconocibles por autómatas de pila).

Ejemplo práctico en notación BNF estándar:

Gramática para $L = \{anbn \mid n \geq 1\}$ $L = \{anbn \mid n \geq 1\}$.

```
<S> ::= a <S> b | a b
```

Tipo 3 – Gramáticas regulares (lenguajes regulares)

Definición: Son un subconjunto de las gramáticas libres de contexto. Pueden ser:

- Lineales por la derecha: $A \rightarrow wB$ $A \rightarrow wB$ o $A \rightarrow w$ $A \rightarrow w$, donde ww es una cadena de terminales (puede ser vacía) y A, BA, B son no terminales.
- O lineales por la izquierda: similar pero con $A \rightarrow Bw$ $A \rightarrow Bw$.

Lenguaje generado: Lenguajes regulares (reconocibles por autómatas finitos).

Ejemplo práctico en notación BNF estándar:

Gramática para el lenguaje $L = \{anbm \mid n \geq 1, m \geq 0\}$ $L = \{anbm \mid n \geq 1, m \geq 0\}$ (una o más a's seguidas de cero o más b's).

```
<S> ::= a <A>
<A> ::= a <A> | b <B> | b
<B> ::= b <B> | ε
```

Ahora bien, la relación gramática–lenguaje en cada caso es la misma: el lenguaje es el conjunto de todas las cadenas de terminales derivables desde el símbolo inicial aplicando las reglas. La diferencia radica en la forma permitida de esas reglas y, por tanto, en la complejidad de los lenguajes que se pueden describir.

2.Representación Estructurada de Árboles de Derivación (GLC)

Este documento contiene la especificación formal de la Gramática Libre de Contexto orientada a herramientas de dibujo sobre el alfabeto $\Sigma = \{a, c, g, t\}$, junto con los árboles de

sintaxis abstracta o de derivación para las estructuras complejas solicitadas (Árboles y Figuras).

Definición de la Gramática $G = (V, \Sigma, P, S)$:

- $V = \{S, A, B, C, D, T\}$
- $\Sigma = \{a, c, g, t\}$ donde: a (avanzar/línea), c (esquina 90°), g (giro/bifurcación), t (trazo/hoja)
- P (Reglas de Producción):
 1. $S \rightarrow C \mid B \mid D$
 2. $C \rightarrow acacacac$ (Cuadrado)
 3. $B \rightarrow CgCgC$ (Cubo)
 4. $D \rightarrow aDa \mid gTg \mid t$ (Ramas/Hojas del Árbol)
 5. $T \rightarrow tT \mid t$ (Hojas continuas)

1. Árbol de Derivación: Árbol Pequeño (Tallo y Hoja)

Representa la derivación de la cadena 'ata' mediante la sustitución sintáctica de la variable estructural del árbol.

Estructura del Árbol (Cadena: "ata"):

```

      S
      |
      D
    /  |  \
   a  D  a
      |
      t
  
```

2. Árbol de Derivación: Árbol con Ramas Laterales

Representa la derivación de la cadena 'agttga' incorporando la variable de bifurcación T para simular el follaje lateral.

Estructura del Árbol (Cadena: "agttga"):

```

      S
      |
      D
    /  |  \
   a  D  a
      |  |  \
     g  T  g
      /  \
  
```


t T
|

3. Árbol de Derivación: Árbol Frondoso Complejo

Muestra el crecimiento recursivo anidado de la estructura del árbol para la cadena 'aagttgaa'.

Estructura del Árbol (Cadena: "aagttgaa"):

S
|
D
/|\
a D a
/|\
a D a
/|\
g T g
/\
t T
|
t

4. Árbol de Derivación: Cuadrado Estándar

Representa el despliegue secuencial de los comandos del alfabeto para trazar una figura geométrica bidimensional cerrada ('acacacac').

Estructura del Árbol (Cadena: "acacacac"):

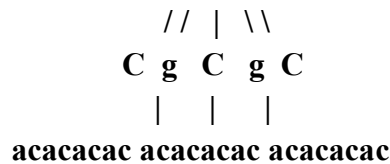
S
|
C
//// \\\\
a c a c a c a c

5. Árbol de Derivación: Cubo Tridimensional

Demuestra la composición modular de figuras complejas conectadas por giros libres ('acacacacgacacacgacacac').

Estructura del Árbol (Estructura del Cubo):

S
|
B
//|\\
// |\\



Patologías de las Gramáticas

El diseño correcto de una gramática formal es un requisito para el funcionamiento robusto de los analizadores sintácticos en los compiladores. Las gramáticas mal estructuradas o diseñadas presentan defectos importantes que pueden causar errores críticos, ya sea permitiendo múltiples interpretaciones de un mismo código fuente o induciendo a los analizadores a entrar en bucles infinitos (Aho et al., 2007; Johnson y Zelenski, 2012). Para garantizar el correcto procesamiento de los lenguajes de programación, es necesario someter estas gramáticas a procesos de higiene y optimización. A continuación, se detallan tres de los defectos más comunes junto con casos prácticos de resolución.

Gramática Ambigua

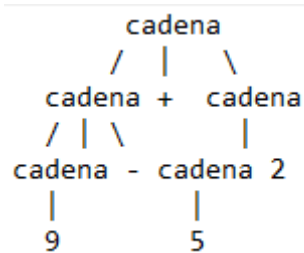
Se define que una gramática es ambigua si permite construir más de un árbol de derivación (o *parse tree*) para generar exactamente la misma cadena de entrada (Aho et al., 2007). Esta multiplicidad de árboles implica que la cadena de texto puede tener más de un significado o interpretación semántica, lo cual resulta inaceptable en el contexto de los lenguajes de programación, donde la ejecución debe ser determinística y exacta (Johnson y Zelenski, 2012).

Un caso práctico clásico de ambigüedad ocurre en las expresiones aritméticas sin reglas estrictas de precedencia ni asociatividad. Considere la siguiente gramática simplificada:

$$cadena \rightarrow cadena + cadena \mid cadena - cadena \mid 0 \mid \dots \mid 9$$

Si se establece una secuencia de entrada (por ejemplo, $9 - 5 + 2$), esta gramática permite demostrar la ambigüedad al poder generar dos árboles de derivación distintos para la misma entrada:

Primero, para representar la Asociatividad por la izquierda, el nodo raíz deriva en la regla " $cadena + cadena$ ". La primera *cadena* vuelve a derivar en " $cadena - cadena$ " (reemplazándose por sus *hojas* 9 y 5 respectivamente), mientras que la *segunda cadena* (después del $+$) se reemplaza por la *hoja* 2. Esto equivale a la agrupación " $(9 - 5) + 2$ ", cuyo resultado matemático de la evaluación es 6.

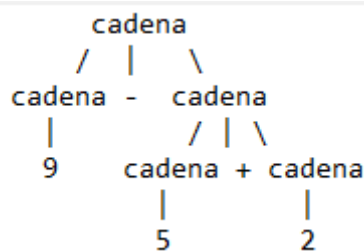


$$(9 - 5) + 2$$

$$(4) + 2$$

Resultado = 6

A hora, si se quiere representar un árbol aplicando Asociatividad por la derecha, el nodo raíz deriva en la regla “cadena - cadena”. La *primera cadena* se convierte directamente en la *hoja 9*. La *segunda cadena* (después del -) “cadena + cadena” deriva en (reemplazándose por sus *hojas 5 y 2*). Esto equivale a la agrupación “9 - (5 + 2)”, cuyo resultado erróneo en la evaluación es 2.



$$9 - (5 + 2)$$

$$9 - (7)$$

Resultado = 2

La ambigüedad no es un error de sintaxis en el código fuente, se considera una falta de precisión en las reglas de la gramática. Si una cadena puede generar dos árboles de derivación diferentes, el compilador pierde su naturaleza determinista. Siguiendo los ejemplos: un árbol evaluará la operación con asociatividad por la izquierda (**resultado = 6**) y el otro por la derecha (**resultado = 2**). Para solucionar esto, es obligatorio reestructurar la gramática introduciendo niveles de jerarquía que fuercen matemáticamente la precedencia y la asociatividad de los operadores, garantizando una única interpretación semántica exacta.

Recursividad por la Izquierda

Se dice que una producción gramatical es recursiva por la izquierda si el símbolo no terminal de la cabeza (lado izquierdo) es exactamente el mismo que el primer símbolo de su cuerpo (lado derecho) (Moreno, 2004). Formalmente, existe recursividad por la izquierda si se puede realizar una derivación en uno o más pasos del tipo $A \Rightarrow +A\alpha$ (Aho et al., 2007). Los analizadores sintácticos descendentes (top-down) no pueden procesar este tipo de gramáticas ya que entran rápidamente en un bucle infinito al intentar expandir el mismo no terminal repetidamente sin lograr consumir ningún símbolo de entrada (Johnson y Zelenski, 2012).

Para eliminar esta patología, se aplica un algoritmo de optimización paso a paso. El método básico consiste en tomar las producciones con recursividad inmediata de la forma $A \rightarrow A\alpha|\beta$ (donde β es una secuencia que no comienza con A) y reemplazarlas por producciones equivalentes que utilizan recursividad por la derecha introduciendo un nuevo no terminal A' (Aho et al., 2007; Vidyalay, 2020):

1. $A \rightarrow \beta A'$
2. $A' \rightarrow \alpha A'|\epsilon$

Caso práctico: sea la gramática para derivar la suma de expresiones: $\mathcal{G}$

- Paso 1: se identifican los componentes del problema según la fórmula de la patología. El *no terminal* es $A = E$, la *secuencia recursiva* es $\alpha = +T$, y la *alternativa no recursiva* es $\beta = T$.
- Paso 2: se aplica la sustitución aislando el problema e introduciendo el nuevo no terminal E' .

$$A \rightarrow \beta A' \Rightarrow E \rightarrow TE'$$

$$A' \rightarrow \alpha A'|\epsilon \Rightarrow E' \rightarrow +TE'|\epsilon$$

- Resultado optimizado: la nueva gramática equivalente e higienizada es: $E \rightarrow E + T \mid T$
 $E' \rightarrow TE'$
 $E' \rightarrow +TE'|\epsilon$

Esta patología afecta directamente a los analizadores sintácticos descendentes (*top-down*). Cuando una producción tiene la forma $A \rightarrow A\alpha$, el algoritmo del *parser* intenta expandir el símbolo de la cabeza antes de leer o consumir cualquier token real de la entrada; al no avanzar en la cadena de texto, el compilador cae en una recursión infinita en memoria. La solución algorítmica consiste en transformar la recursividad hacia la derecha introduciendo un símbolo no terminal auxiliar A' y una producción vacía (ϵ). Al aplicar esto a

expresiones como $E \rightarrow E + T|T$, se obtiene que el *parser* posponga la evaluación recursiva hasta haber procesado el primer término válido, rompiendo el ciclo infinito.

Factorización por la Izquierda

La factorización por la izquierda es otra de las transformaciones gramaticales indispensables para producir gramáticas aptas para el análisis predictivo descendente. Esta patología se presenta cuando dos o más alternativas de producción para un mismo no terminal comparten un prefijo común prolongado. Al encontrar este prefijo, el analizador sintáctico no tiene forma de decidir qué regla de producción debe aplicar simplemente mirando el siguiente símbolo (*lookahead*) de entrada (Aho et al., 2007).

La optimización consiste en abstraer (factorizar) el prefijo común en una regla principal compartida y diferir la decisión de la divergencia hacia un nuevo no terminal (Johnson y Zelenski, 2012). Si se tienen producciones de la forma $A \rightarrow \alpha\beta_1|\beta_2$, la gramática resultante optimizada será:

1. $A \rightarrow \alpha A'$
2. $A' \rightarrow \beta_1|\beta_2$

Caso práctico: esta patología es muy común en el problema del "else colgante" (*dangling-else*) al modelar las instrucciones condicionales (Aho et al., 2007):

$Stmt \rightarrow \text{if } (Cond) \text{ then } Stmt$
 $\quad | \text{if } (Cond) \text{ then } Stmt \text{ else } Stmt$
 $\quad | a$

Aquí, las dos primeras alternativas del no terminal $Stmt$ comparten el prefijo largamente común $\text{if } (Cond) \text{ then } Stmt$. Si aplicamos el algoritmo de la factorización por la izquierda para optimizar esta gramática gramatical y erradicar el conflicto, obtenemos:

$Stmt \rightarrow \text{if } (Cond) \text{ then } Stmt \text{ OptElse } | a$
 $OptElse \rightarrow \text{else } Stmt | \epsilon$

Con esta gramática resultante optimizada, el analizador puede procesar tranquilamente la porción principal del bloque $\text{if } (Cond) \text{ then } Stmt$ y luego, evaluando el siguiente símbolo, decidir fácilmente si aplica la rama extendida con el else o si finaliza asumiendo la cadena vacía ϵ .

Esta patología ocurre cuando el *parser* se encuentra ante una encrucijada de decisiones. Si múltiples reglas para un mismo no terminal comparten un prefijo común

extenso (como ocurre en las estructuras condicionales *if (Cond) then...* con o sin *else*, el analizador no puede saber qué camino tomar mirando únicamente el siguiente símbolo disponible (*lookahead*).

Para eliminar esta indecisión del compilador, se aplica una abstracción similar al factor común en álgebra. El prefijo común se extrae en una regla principal y la bifurcación (el desvío) se delega a un nuevo no terminal (*OptElse*). Esto permite que el *parser* consuma la estructura idéntica con total certeza y decida de manera determinista la existencia de la rama opcional evaluando el token posterior.

4. De la Expresión al Autómata (Caso Práctico: Ajedrez).

4.1 Notación PGN (Portable Game Notation)

La Notación Portátil de Partidas (PGN, por sus siglas en inglés) es un estándar de formato de texto plano para el registro de partidas de ajedrez, diseñado para ser legible tanto por humanos como por programas informáticos. Un archivo PGN típico comienza con un conjunto de pares de etiquetas que describen metadatos de la partida (evento, lugar, fecha, jugadores, resultado, etc.), seguido del texto de movimientos (movetext), que contiene la secuencia de jugadas en notación algebraica.

El reconocimiento de un lenguaje tan estructurado como el PGN puede abordarse mediante autómatas finitos, que son modelos matemáticos de computación con memoria limitada, ideales para el análisis léxico o de patrones. Para este caso práctico, se plantea un autómata finito que reconozca un subconjunto del lenguaje PGN, específicamente la estructura de los pares de etiquetas y una secuencia simplificada de movimientos.

4.1.1 Autómatas Finitos

El siguiente autómata finito determinista (AFD) reconoce la estructura general de un archivo PGN que contiene pares de etiquetas y una lista de movimientos.

- **Alfabeto (Σ):** Caracteres ASCII imprimibles relevantes: [., ", letras (A-Z, a-z), números (0-9), espacio, -, /, :, y los caracteres específicos de los movimientos (O, -, x, =, +, #, *).
- **Estados (Q):**
 - q0: Estado inicial.
 - q1: Leyendo el nombre de una etiqueta.
 - q2: Leyendo el valor de una etiqueta (entre comillas).
 - q3: Estado de transición después de una etiqueta.
 - q4: Leyendo el texto de movimientos (movetext).
 - q5: Estado de aceptación (archivo PGN válido).

- **Estado inicial:** q0
- **Estados de aceptación:** q5
- **Función de transición (δ):** (Se presentan las transiciones principales)

Estado	Símbolo de entrada	Siguiente estado
q0	[	q1
q1	Letra, -	q1
q1	“	q2
q2	Todo menos “	q2
q2	“	q3
q3	]	q0
q0	Letra, número, 0, -, x, =, +, #, *, espacio	q4
q4	Letra, número, 0, -, x, =, +, #, *, espacio, .	q4
q4	Fin de cadena	q5

Primero se identifican dos componentes principales de un archivo PGN, que serían los pares de etiquetas y el texto de movimientos para después crear estados para representar cada fase del análisis (Inicio, lectura del nombre, lectura del valor, cierre de etiqueta y lectura de movimientos). Una vez realizado esto, se definen las reglas de transición basadas en la sintaxis PGN.

Es decir, una etiqueta comienza con [, seguido de un nombre que puede estar en letras y usar guiones, luego comillas (“), el valor, comillas (“) y finalmente cierre de corchetes (]). Después de las etiquetas, cualquier carácter que no sea un corchete se considera parte del movetext. Con este autómata podemos proporcionar una base para el reconocimiento de un archivo PGN.

4.2 Definición de un subconjunto simplificado de PGN

Para facilitar el diseño de la expresión regular y el autómata, se define un subconjunto del lenguaje PGN que abarca los movimientos básicos más comunes en notación algebraica estándar (SAN). Este subconjunto excluye movimientos complejos como la promoción, el enroque y las anotaciones de jaque/mate, centrándose en la estructura fundamental de un movimiento.

- **Movimiento:** [Pieza] [Desambiguación] [Captura] Destino

- **Pieza:** K (Rey), Q (Dama), R (Torre), B (Alfil), N (Caballo). Es opcional para los peones.
- **Desambiguación:** Archivo (a-h) y/o rango (1-8) para distinguir entre dos piezas iguales que pueden moverse a la misma casilla.
- **Captura:** x (indica que se captura una pieza).
- **Destino:** Coordenada de la casilla final, compuesta por una letra (a-h) y un número (1-8).

4.3 Diseño de la expresión REGEX y el Autómata Finito Determinista Equivlanete

4.3.1 Expresión regular para el Subconjunto simplificado

Basándose en la gramática definida, se debe construir la siguiente expresión regular para reconocer un movimiento individual:

$$^([KQRBN])?([a-h])?([1-8])?(x)?([a-h][1-8])\$$$

Primero se identifican los componentes, descomponiendo la estructura del movimiento en sus elementos fundamentales, es decir: Pieza, desambiguación, captura y destino. En este caso se utilizaron clases de caracteres ([KQRBN], [a-h], [1-8]) y cuantificadores (“?” Para opcional) para representar cada componente. Una vez realizado esto, se concatenan los patrones en el orden correcto para formar la expresión regular completa.

4.3.2 Autómata Finito Determinista (AFD) Equivalente.

Este AFD se diseña para reconocer exactamente el mismo lenguaje definido por la expresión regular. Los estados representan el progreso en el reconocimiento de los componentes del movimiento.

- **Alfabeto (Σ):** {K, Q, R, B, N, a-h, 1-8, x}
- **Estados (Q):**
 - q0: Estado inicial (esperando inicio del movimiento).
 - q1: Se ha leído una pieza o se está en la fase de desambiguación.
 - q2: Se ha leído la desambiguación de archivo.
 - q3: Se ha leído la desambiguación de rango.
 - q4: Se ha leído el símbolo de captura (x).
 - q5: Se ha leído la primera parte de la casilla de destino (letra).
 - q6: Estado de aceptación (se ha leído un movimiento completo).
 - q_error: Estado de error (para simplificar, las transiciones no definidas llevan aquí).
- **Estado inicial:** q0
- **Estado de aceptación:** q6

Estado	Símbolo de entrada	Siguiente estado	Observación
q0	K, Q, R, B, N	q1	Se ha leído una pieza.
q0	a-h	q5	Movimiento de peón.
q0	Otro	q_error	
q1	a-h	q2	Desambiguación de archivo.
q1	1-8	q3	Desambiguación de rango.
q1	x	q4	Captura.
q1	a-h	q5	Inicio de la casilla de destino (Sin desambiguación ni captura)
q2	1-8	q3	Desambiguación de rango.
q2	x	q4	Captura.
q2	a-h	q5	Inicio de destino (sin rango ni captura)
q3	x	q4	Captura.
q3	a-h	q5	Inicio de destino.
q4	a-h	q5	Inicio de destino después de captura.
q5	1-8	q6	Fin del movimiento (Destino completo)
q6	-	-	Estado de aceptación.

El AFD se construyó siguiendo la estructura de la expresión regular. Cada estado del AFD corresponde a un punto específico en el proceso de emparejamiento de la regex. Para el manejo de opcionales se tradujeron en múltiples transiciones desde un mismo estado, permitiendo que el autómata decida si consumir o no esos caracteres.

El autómata sigue el orden de los componentes: Pieza (opcional), desambiguación (opcional), captura (opcional) y destino (opcional). Por cada estado y símbolo de entrada, solo hay una transición posible, por lo que se cumple con la definición del AFD.

CONCLUSION

La investigación y el desarrollo práctico presentados en este documento demuestran de manera contundente que la Teoría de Lenguajes y Gramáticas Formales no es meramente un constructo abstracto, sino la infraestructura matemática que sostiene la ingeniería de software y el diseño de compiladores modernos. Mediante la exploración sistemática de la Jerarquía de Chomsky, se ha podido evidenciar cómo las restricciones de producción dictan directamente la complejidad computacional y el tipo de autómata necesario para el procesamiento de un lenguaje.

La aplicación práctica de las Gramáticas Libres de Contexto (GLC) enfocadas en modelado geométrico, junto con sus respectivos árboles de derivación, ilustra el poder de los sistemas generativos para estructurar comandos secuenciales y anidados. Por otro lado, el estudio de las patologías gramaticales (ambigüedad, recursividad e indecisión por prefijos comunes) resalta que un compilador eficiente no solo requiere reglas válidas, sino un proceso riguroso de higiene y optimización algorítmica. Sin transformaciones como la eliminación de recursividad por la izquierda o la factorización por la izquierda, los analizadores sintácticos *top-down* estarían irremediablemente condenados a fallos lógicos o recursiones infinitas en memoria.

Finalmente, el traslado de la teoría de lenguajes regulares al ámbito léxico mediante el subconjunto simplificado de la notación PGN de ajedrez comprueba la viabilidad de los Autómatas Finitos Determinísticos (AFD) y las expresiones regulares para el reconocimiento exacto y determinista de patrones estructurados. En conclusión, para el estudiante de Ingeniería en Informática, el dominio de estos mecanismos de traducción, modelado e higienización es la piedra angular que transforma la programación empírica en verdadera arquitectura computacional robusta y determinista.

BIBLIOGRAFÍA

Aho, A., Lam, M., Sethi, R. y Ullman, J. (2007). *Compilers: Principles, techniques, and tools* (2da ed.). Pearson Addison-Wesley. https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Alfred-V.-Aho-Monica-S.-Lam-Ravi-Sethi-Jeffrey-D.-Ullman-Compilers-Principles-Techniques-and-Tools-Pearson_Addison-Wesley-2007.pdf

Johnson, M., y Zelenski, J. (2012). *Formal grammars* [Material de clase]. Stanford University. <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/080%20Formal%20Grammars.pdf>

Moreno, M. (2004). *Elimination of left recursion* [Material de clase]. University of Western Ontario. <https://www.csd.uwo.ca/~mmorenom/CS447/Lectures/Syntax.html/node8.html>

Vidyalay, G. (2020). *Left recursion | Left recursion elimination*. <https://www.gatevidyalay.com/tag/left-recursion-elimination-example/>

Edwards, S. J. (1993). *Portable Game Notation Specification and Implementation Guide*. Recuperado de <http://www.chessclub.com/help/PGN-spec> (como se cita en Johansson, 2011)

Johansson, O. (2011). *Regexp::Common::Chess - regexp for algebraic notation in chess* (Versión 0.1) [Módulo Perl]. CPAN. Recuperado de <https://metacpan.org/pod/Regexp::Common::Chess>

PGN::Move. (s.f.). En *RubyDoc.info*. Recuperado de <https://rubydoc.info/gems/pgn/0.2.0/PGN/Move>

Portable Game Notation. (2023, diciembre 12). En *Wikipedia, La enciclopedia libre*. Recuperado de https://es.wikipedia.org/wiki/Notaci%C3%B3n_port%C3%A1til_de_partida

Portable Game Notation. (s.f.). En *Wikipedia*. Recuperado de https://en.wikipedia.org/wiki/Portable_Game_Notation